WHAT EVERYONE SHOULD KNOW ABOUT SCHEDULING COMPUTATIONS

PAPP, P. A., YZELMAN, A. N., STEINER, R. S., AND ...

Abstract. Scheduling tasks on compute systems of any scale has become ubiqutous, and is no longer a specialist topic in computer science and applied mathematics. Over the past few years, Huawei has endeavoured to further develop the theoretical foundations of scheduling, crucially from the perspective that in modern compute systems, data movement is by far more costly than actual computation. This industry-wide trend has wide-ranging practical implications– not only in areas ranging from parallel system to algorithm design, but also on software, communication layers, and run-time systems. The article before you summarises our recent advances, outlines their practical implications, and translates these to concrete advice on parallel system and software design.

1. Introduction

scheduling is everywhere: CPUs, NPUs, SOCs
scheduling becomes more intricate at scale:

- multi-die,
- multi-device,
- cloud & supercomputer scale

computation as a (Hyper)DAG
scheduling is not partitioning

1.1. Computation model.

PRAM is not sufficient, must cost communication
note that ideally, furthermore: hierarchical

- hierarchical processor designs, hierarchical nodes, hierarchical networks

latencies matter (can cite Anastasiadis if ready)

1.2. Applications.

a summary of where scheduling is applied at present

- run-time systems, such as OpenMP;
- communication layers, such as MPI, GASNET, others (PRAM/PGAS?);
- within compilers (to various degrees)
- offline system design (e.g., Wireless)
- not meant as an exhaustive list of applications, but rather to index the quite different domains of applicability

2. Scheduling is hard

It has been long known that scheduling is a particularly challenging problem. The complexity of problems is computer science is often defined via the concept of NP-hardness. Intuitively speaking, an NP-hard problem is one that is very unlikely to have an algorithm that can always find the optimal solution efficiently (i.e. with a running time that is polynomial in the size of the problem). In particular, if there was an efficient algorithm for any of the NP-hard problems, then all the other NP-hard problems could also be solved efficiently by reducing these problems to each other. There are thousands of problems that are known to be NP-hard, and many of these have been extensively studied by the community for decades; as such, when a problem is NP-hard, it is generally accepted that no polynomial-time algorithm exists for it.

Besides NP-hardness, there are other concepts that show that the problem is even more challenging; for instance, inapproximability means that it is even NP-hard to find a solution that is close to the optimal value, whereas parameterized complexity studies whether the problem remains hard if some parameters of the input are fixed to a small value.

With the above tools, it has been shown that scheduling is a hard problem even in very simple cases: with unit execution time for all tasks and unit communication delays between the processors, finding

Date: 25th March 2026.

the optimal schedule is still NP-hard [RS87]. In more realistic settings, like the BSP model, the problem is even more difficult. For instance, the optimal solution is still NP-hard when considering very special classes of DAGs, such as DAGs with only two layers, or in-trees where each vertex has at most one outgoing edge. Moreover, in general DAGs, the optimal cost is even NP-hard to efficiently approximate beyond a specific ratio $(1 + \delta)$, for some $\delta > 0$; in official terms, the problem does not allow for a so-called Polynomial-Time Approximation Scheme. In fact, in BSP scheduling, even if the assignment of nodes to processors and supersteps is already fixed, the subproblem of scheduling just the necessary communication steps into the communication phases is already NP-hard [PAY25].

On the positive side, some theoretically efficient (polynomial-time) algorithms do exists for edge cases, e.g. when the DAG only consists of independent chains and the number of processors is small, but these algorithms are also very inefficient in practice [PAY25].

TODO: [layman terms needed here?]

2.1. **Two-level scheduling.** NP-, APX-hardness, counter-examples, arbitrary factors away from optimum

2.2. **Vertical data movement: communication *and* I/O.** Another vital aspect of scheduling computations in practice is vertical data movement. This concept of not included in the base BSP model for simplicity: BSP only considers the cost of moving data horizontally from the memory associated with one processor to memory associated with another. However, when executing real computations (even on just a single processor), one of the most important factors is that fast memory in the system (e.g. cache) often has very limited capacity, whereas we also have some other memory (e.g. RAM) with drastically larger capacity, but using this is much slower. In particular, the cost of moving data from slow memory to fast memory (or vice versa) is often the bottleneck in the execution of complex computations.

This vertical communication cost (also known as I/O cost) is most often analyzed in the so-called red-blue pebble game of Hong and Kung [HK81]. This is essentially a single-processor scheduling problem in a two-level memory hierarchy (with e.g. cache and RAM), where the goal is to minimize the I/O operations while executing the computations. The red-blue pebble game has been widely used to analyze the I/O cost of algorithms and also to develop I/O lower bounds for many important computations, such as matrix multiplication, fast Fourier transform or attention in neural networks [HK81, SY24]. However, in general DAGs, finding the optimal I/O cost is again a problem that is NP-hard to even approximate to any reasonable factor [BPY25].

For a more realistic model of paralellizing computations, one needs to combine the BSP model, which allows multiple processors and horizontal communication, with this red-blue pebble game that captures memory limitations and I/O costs for the single-processor case. Simply generalizing the red-blue pebble game to a parallel setting results in a multi-processor red-blue pebbling model [BPY25], whereas if we also incorporate vertex weights and synchronization costs from BSP, we obtain a more complex Multi-BSP (or MBSP) model [PBY25]. These combinations offer much more realistic models for the scheduling problem, but on the other hand, finding the optimal solution becomes even more challenging. In particular, the hardness results for both BSP scheduling and red-blue pebbling carry over to these combined models [BPY25].

However, even if the optimal schedules are essentially impossible to find, the theoretical and experimental study of these models still offers valuable insights. For instance, one can analyze the difference between holistic scheduling strategy that aims to find a solution for the entire problem, and a two-stage approach that first comes up with a good parallelization strategy, and then combines this with a cache management strategy to optimize I/O costs. On the theoretical side, one can show a worst-case example where the two-stage schedule is a very large factor away from the actual optimum, even if both stages are optimal separately. On the experimental side, one can confirm that the holistic view can indeed outperform the two-stage approach by a small margin [PBY25].

TODO: [Savage needed?]

2.3. **Partial computations.** The multi-processor red-blue pebbling or MBSP models above already capture most of the key factors of scheduling a computation. However, there are of course many further aspects that could be incorporated into the model to make it even more realistic. Here we briefly discuss one of these extensions: partial computations.

For simplicity, the original red-blue pebble game assumes that every operation is computed in one go, with all of its inputs in fast memory simultaneously. However, if the operation is e.g. the addition or multiplication of numerous inputs, then in practice, it can also be executed in multiple steps: we first aggregate only some of the input values, maybe even save this partial results into slow memory and load

it back later, then continue with another subset of the inputs, and so on. Such a partial computing strategy might allow for a lower I/O cost in some cases, or it might even be necessary to allow a feasible solution when an operation has a very high number of inputs.

The red-blue pebble game can naturally be extended with such partial computations to allow for a more realistic model of I/O costs [PSY25, Sob26]. From a theoretical perspective, this extended model is rather similar to the original game in terms of e.g. inapproximability. Moreover, one of the most important tools related to red-blue pebbling is so-called S-partitions, which were widely used to derive lower bounds on matrix multiplication, fast Fourier transform, neural network attention, and many other computations [HK81, SY24]. These S-partitions and the corresponding proofs can also be adjusted to the partial computing model, which allows to derive the same I/O lower bounds with partial computations (up to constant factors) for the example computations mentioned above. TODO: [state the concrete bounds? it would require introducing a LOT of notation...]

However, we also emphasize that in general, partial computations can enable significantly different pebbling strategies. For instance, they can notably reduce the optimal I/O cost in sparse matrix-vector multiplications, or even by a linear factor in some synthetic examples [PSY25].

TODO: [Determining optimal schedules using ILP formulations at least as hard as the non-partial case (?) - needed?]

3. Avoiding hardness

While general scheduling while taking into account realistic costs is hard as outlined in the preceding section, scheduling in practice works around hardness in a variety of ways. We index common approaches along the following practical use cases:

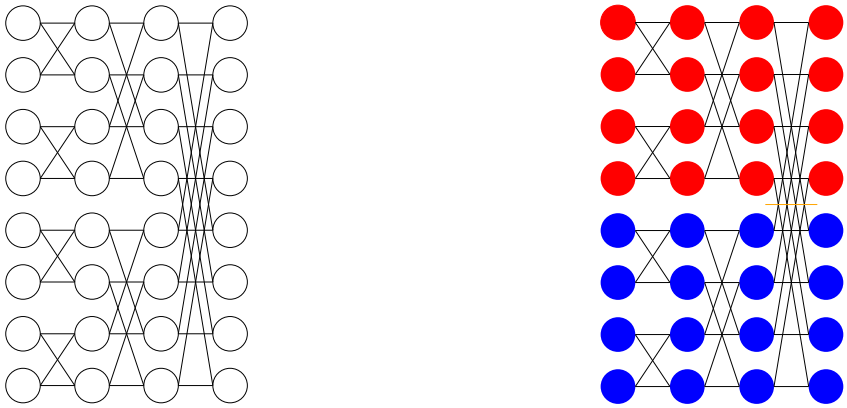
(1) minimise horizontal data movement (communication);
(2) minimise vertical data movement (I/O); and
(3) minimise data movement through NUMA hierarchies.

We focus on proven techniques by which the high-performance computing (HPC) community has operated in the last few decades, which principially considers proving optimality for specific problems or specific classes of problems, and discovering parallel algorithms that achieve these bounds. This approach stands in stark contrast to finding solutions that work in the most generic cases. We couple outlining these state-of-the-art methodologies with novel insights from our research, and discuss practical implications to parallel frameworks for AI and beyond, run-time system design for modern large-scale compute infrastructures, and insights tailored to the complex case of irregular computations.

3.1. Horizontal data movement.

Minimisation of horizontal data movement, i.e., communication minimisation, comes naturally when adopting a model of computation that assigns explicit costs to data movement and synchronisation. One such model, the Bulk Synchronous Parallel (BSP) model, comes with two modes of operation, one automatic and one manual [Val90]. In both cases, the cost model revolves not only around work, but also around supersteps and h-relations that together quantify the cost of data movement– see Section 1.1 for details. Costs are typically parametrised in the number of processors p and some problem-specific parameter n, such as a problem size. The key phrase here is *problem-specific*: in HPC, by far most known algorithms are manual-mode BSP, and literature is rich with examples of optimal algorithms or algorithms that operate within an optimal trade-off regime. We give two examples, fast Fourier transformation (FFT) and matrix multiplication, and demonstrate how optimal algorithms and optimal trade-offs may be derived.

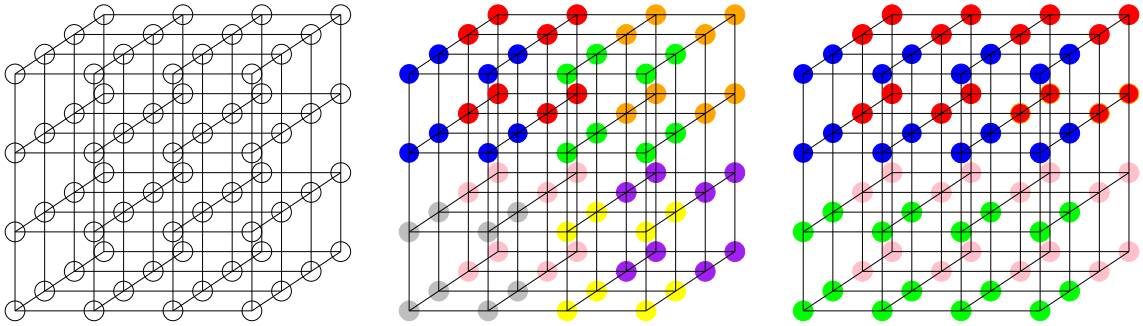
Fast Fourier transforms and butterfly DAGs. Consider the visual representation of a one-dimensional FFT of length $n = 8$ depicted in Figure 3.1a. This so-called *butterfly graph* consists of three phases, and of $\log n$ phases for general n. When distributing the computation over multiple processors, different vertices are assigned to different processes. Edges between vertices assigned to different processors induce communication between processes. By cutting the graph horizontally, we visually confirm the computation may proceed embarrasingly parallel for the first phase up to $p \leq 4$, while the first two phases may be computed concurrently for up to $p \leq 2$ processors. Figure 3.1b visualises the latter, from which we observe that this particular DAG partitioning results in exactly n cut edges. The communication from phase two to three furthermore is balanced amongst the processors– the communication cost hence is proportional to n/p, while the number of supersteps is one and processor-local work is proportional to $(n \log n)/p$. The above arguments that reason about a fixed butterfly DAG structure extend to an achievable lower bound for general n [PY88, Val90, IB01].



(A) Directed acyclic graph representation of a fast Fourier transform on an input of length eight. Connections should be read from left to right.

(B) A two-processor parallelisation assigns vertices to a blue or red processor. An orange line indicates cut edges that require communication.

FIGURE 3.1. Butterfly graphs and their application to fast Fourier transforms.



(A) The computational DAG for multiplying two 4×4 matrices.

(B) Communication-optimal work distribution over eight processors.

(C) Memory-optimal work distribution over four processors.

FIGURE 3.2. Cube graphs of size $4 \times 4 \times 4$ and their application to matrix–matrix multiplication.

Given that the sequential FFT algorithm has a work cost proportional to $n \log n$ –and any computation that maps to a butterfly graph a work cost that is *at least* proportional to $n \log n$– the total parallelisation cost of $(n \log n)/p$ is optimal[1], and any computation that maps to a butterfly graph can be optimally parallelised by this singular lower-bound analysis and parallelisation recipe. Colloquially, algorithms backed by (or resulting from) such proofs of optimality are known as *immortal* algorithms.

Matrix–matrix multiplication. Let us consider two matrices A, B both of size $n \times n$, and the matrix–matrix multiplication $C = AB$. Figure 3.2a depicts the resulting DAG for $n = 4$, where, for example, the matrix A could span the horizontal and vertical axes (the front of the cube), the matrix B the horizontal and depth axes (the top of the cube), and the output matrix C the vertical and depth axes (the side of the cube)– in which case the direction of the horizontally-oriented edges would face towards C, while the depth- and vertically-oriented edges would face *away* from A and B, respectively. All vertices shown correspond to multiplications $a_{ik}b_{kj}$ of the canonical triple-loop matrix–matrix multiplication algorithm, which, along the edges towards C, are summed into the output c_{ij}.

In the same vein as for the FFT, we may construct a parallel algorithm by exploiting the geometric features of the resulting cube DAG. While coloring vertices to specific processors, again edges between differently-coloured nodes will induce communication. Therefore the work distribution that minimises communication is one that minimises such cuts, which in the case of a cube means identifying p volumes for which the surface areas (where edge cuts occur) are minimised. Figure 3.2b exemplifies such a distribution, which achieves a communication cost proportional to $(n/p^{1/3})^2 = n^2/p^{2/3}$ in two supersteps: first, every input from A and B are spread out over $p^{1/3}$ processors while second, every output in C is

[1]assuming the work associated with each vertex has homogeneous cost and that p is large enough compared to n

accumulated from partial results of $p^{1/3}$ processors. In-between, a process-local sequential multiplication takes place that costs $(n/p^{1/3})^3 = n^3/p$ work per processor, which is work-optimal as well. The above geometric argument is extensible to a formal proof of optimality[2] in terms of horizontal data movement for general n, and practical algorithms that achieve this bound exist [DNS81, Ber89, ACS90, MT99, ITT04].

Parametrisation and poly-algorithms. The resulting so-called 3D algorithm for matrix multiplication, however, is not an immortal algorithm: while communication- and work-optimal, the processor-local memory requirement is proportional to $n^2/p^{2/3}$– larger than the ideal bound of n^2/p. Subdividing the matrices A, B^T, and C into p square blocks results in the so-called 2D algorithms, which are work- and memory-optimal with costs proportional to n^3/p and n^2/p, respectively. Such algorithms are not communication-optimal, however, as processors need to communicate $\sqrt{p} - 1$ blocks of size n^2/p over $\sqrt{p} - 1$ supersteps to maintain memory optimality. Figure 3.2c depicts the resulting work distribution for $n = p = 4$, which visualises a higher surface-to-volume ratio compared to the 3D algorithm, leading to a suboptimal communication cost.

The existence of both 2D and 3D algorithms points to a discrete trade-off space between optimality in communication and memory, indeed confirmed through lower-bound studies [ITT04]. This, in turn, identifies an opportunity for programs to, when available, use more memory in order to reduce communication. These decisions may be made automatically at run-time. To realise this, algorithms must first be *parametrised* not only in p, but also in g, l, and available system memory. A thus-parametrised *poly-algorithm* may then dispatch, at run-time, to the best-possible algorithmic variant– whether that is 2D, 3D, or something in-between. In the case of matrix multiplication, an immortal such poly-algorithm, capable of optimally traversing the memory-communication trade-off, is indeed realisable [MT99].

Generalisability. Analysing specific problems and classes of problems through their DAG structure are well-known, tried and tested approaches, applicable to a wide range of problems. Common DAG structures not discussed here include diamond DAGs for e.g. matrix decompositions [Tis02, Pet16, SLV18, BDG+18] and k-ary trees for e.g. broadcasts [PY88, GV94], while the analysis on cube DAGs can straightforwardly be extended to d-dimensional hyper-cubes for capturing, e.g., tensor contractions.

However, applying these approaches requires a human in the loop, oftentimes needing significant expertise. Consider, for example, multi-dimensional FFTs, for which the de-facto standard today applies 1D FFTs in different directions, requiring data transpositions between directions when executing in parallel [PSFL20]. This again corresponds to butterfly DAGs as in Figure 3.1, and by repeating the argument for 1D FFTs, a lower bound of n/p data movement in 1 superstep again emerges– even for higher-dimensional FFTs. However, a practical algorithm that matches these bounds was only recently discovered, more than three decades later [KB23], and required concepts from category theory to formulate [KBS25].

3.2. **Vertical data movement.** This is a much harder problem, solvable to optimality for severely small problems in the general case

- (point forward to Section on OneStopParallel)

Similar implications therefore apply here: limit to specific problems to find lower bounds, then find algorithms that attain them (e.g., GEMV)

3.3. **NUMA.** be careful with multi-level (distributed) scheduling

gets worse with more NUMA levels (recap SPAA results)

arbitrary factors away from optimal applies when a scheduler has no knowledge about what other schedulers are doing

- therefore, do distributed scheduling where schedulers have a (compressed/sketched) view of other schedulers: *federated scheduling*

Tools for general problem exist: recall Jain & Zaharia, point out weaknesses and differences, point again to OSP for heuristics that work on multi-processor pebbling

3.4. **Parallel frameworks and run-time systems.** novel approaches that automate usage of the above insights, e.g., ALP

A section on general advice for parallel frameworks and run-time systems

[2] again, for large enough n compared to p

3.5. **Irregular computations.** note that irregular problems cannot escape hardness, e.g., SpMV, and point forward to the specific subsection

Index good approaches for problems that are inherently irregular (and thus cannot, to significant degree, avoid hardness)

4. Similar problems, similar results, and solutions

Partitioning, streaming, imply the same work-arounds.

5. OneStopParallel

Describe what OSP contains and aims to provide/support

5.1. **Optimal Scheduling.** Describe ILP-based formulations (don't give the formulations though, keep it high level)

5.2. **Heuristics.** Describe heuristics that are stand-alone and/or part of multi-level / divide-and-conquer approaches in the next section

5.3. **Multi-level.** Describe approaches suitable for cutting up large problems into smaller parts, ILP-based and otherwise

5.4. **Other features.**

6. Results

Summarise key results from the various publications and applications

- EDA / SpTrsv
- MoE
- algorithm-specific parallelisation should be pointed out? (FFT, GEMM, Attention?, etc.)

7. Conclusion

For important kernels or classes of computation, nothing beats manual work (yet:)– lower bounds, trade-off identifications, (poly-)algorithms that achieve them, combined with engineering efforts. Getting bounds though should not be skipped (engineers need to know what to aim for)

Include also mention of OSP-as-a-Service, can deploy in application, or use for testing/exploration

7.1. **Future Work.** Parametric hardness? There was some nice work at SLS26

References

[ACS90] Alok Aggarwal, Ashok K. Chandra, and Marc Snir. Communication complexity of prams. *Theoretical Computer Science*, 71(1):3–28, 1990.

[BDG+18] Grey Ballard, James Demmel, Laura Grigori, Mathias Jacquelin, and Nicholas Knight. A 3d parallel algorithm for qr decomposition. In *Proceedings of the 30th on Symposium on Parallelism in Algorithms and Architectures*, pages 55–65, 2018.

[Ber89] Jarle Berntsen. Communication efficient matrix multiplication on hypercubes. *Parallel Computing*, 12(3):335–342, 1989.

[BPY25] Toni Böhnlein, Pál András Papp, and Albert-Jan N Yzelman. Red-blue pebbling with multiple processors: Time, communication and memory trade-offs. In *International Colloquium on Structural Information and Communication Complexity (SIROCCO)*, pages 109–126. Springer, 2025.

[DNS81] Eliezer Dekel, David Nassimi, and Sartaj Sahni. Parallel matrix and graph algorithms. *SIAM Journal on Computing*, 10(4):657–675, 1981.

[GV94] Alexandros V Gerbessiotis and Leslie G Valiant. Direct bulk-synchronous parallel algorithms. *Journal of parallel and distributed computing*, 22(2):251–267, 1994.

[HK81] Jia-Wei Hong and Hsiang-Tsung Kung. I/O complexity: The red-blue pebble game. In *Proceedings of the 13th ACM Symposium on Theory of Computing (STOC)*, pages 326–333, 1981.

[IB01] Márcia A. Inda and Rob H. Bisseling. A simple and efficient parallel fft algorithm using the bsp model. *Parallel Computing*, 27(14):1847–1878, 2001.

[ITT04] Dror Irony, Sivan Toledo, and Alexander Tiskin. Communication lower bounds for distributed-memory matrix multiplication. *Journal of Parallel and Distributed Computing*, 64(9):1017–1026, 2004.

[KB23] Thomas Koopman and Rob H Bisseling. Minimizing communication in the multidimensional fft. *SIAM Journal on Scientific Computing*, 45(6):C330–C347, 2023.

[KBS25] Thomas Koopman, Rob H Bisseling, and Sven-Bodo Scholz. Category theory for supercomputing: The tensor product of linear bsp algorithms. *arXiv preprint arXiv:2510.00979*, 2025.

[MT99] W. F. McColl and A. Tiskin. Memory-efficient matrix multiplication in the BSP model. *Algorithmica*, 24:287–297, 1999.

[PAY25] Pál András Papp, Georg Anegg, and Albert-Jan N Yzelman. DAG scheduling in the BSP model. In *International Conference on Current Trends in Theory and Practice of Computer Science (SOFSEM)*, pages 238–253. Springer, 2025.

[PBY25] Pál András Papp, Toni Böhnlein, and Albert-Jan N Yzelman. Multiprocessor scheduling with memory constraints: Fundamental properties and finding optimal solutions. In *Proceedings of the 54th International Conference on Parallel Processing (ICPP)*, page 238–247, 2025.

[Pet16] A. Petitet. A bsp scalability analysis of 3d lu factorization with row partial pivoting. SIAM Conference on Parallel Processing for Scientific Computing (PP16), April 2016.

[PSFL20] Doru Thom Popovici, Martin D. Schatz, Franz Franchetti, and Tze Meng Low. A flexible framework for multidimensional dfts. *SIAM Journal on Scientific Computing*, 42(5):C245–C264, 2020.

[PSY25] Pál András Papp, Aleksandros Sobczyk, and Albert-Jan N Yzelman. The impact of partial computations on the red-blue pebble game. In *Proceedings of the 37th ACM Symposium on Parallelism in Algorithms and Architectures*, pages 328–338, 2025.

[PY88] Christos Papadimitriou and Mihalis Yannakakis. Towards an architecture-independent analysis of parallel algorithms. In *Proceedings of the Twentieth Annual ACM Symposium on Theory of Computing*, STOC '88, page 510–513, New York, NY, USA, 1988. Association for Computing Machinery.

[RS87] Victor J Rayward-Smith. Uet scheduling with unit interprocessor communication delays. *Discrete Applied Mathematics*, 18(1):55–71, 1987.

[SLV18] Piyush Sao, Xiaoye Sherry Li, and Richard Vuduc. A communication-avoiding 3d lu factorization algorithm for sparse matrices. In *2018 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, pages 908–919. IEEE, 2018.

[Sob26] Aleksandros Sobczyk. I/o complexity and pebble games with partial computations. *Information Processing Letters*, page 106637, 2026.

[SY24] Barna Saha and Christopher Ye. I/o complexity of attention, or how optimal is FlashAttention? In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024.

[Tis02] A. Tiskin. Bulk-synchronous parallel gaussian elimination. *Journal of Mathematical Sciences*, 108(6):977–991, 2002.

[Val90] Leslie G. Valiant. A bridging model for parallel computation. *Commun. ACM*, 33(8):103–111, August 1990.